

GENRE PACK · 5 LESSONS

H O R R O R

Build a Horror Game That Actually Scares

YOU'LL SHIP

A co-op horror escape with a hunting monster, flashlights, jumpscares and an objective.

- H.1 Atmosphere that scares
- H.2 A flashlight with a battery
- H.3 A monster that hunts
- H.4 Jumpscares that don't lag
- H.5 Objective, escape and rounds

Horror is one of the most-played genres on Roblox, and one of the easiest to get wrong. A dark map with a monster isn't scary on its own. Fear comes from **atmosphere, sound, anticipation and a threat that feels smart**. In this pack you'll build all of that, system by system, into a complete co-op escape game: *Night Shift*.

How to use this pack

- **Before you start:** finish the Core Course up to Module 5 (or know Scripts, LocalScripts, RemoteEvents and attributes).
- **Start a new place:** File → New → Baseplate, then delete the baseplate. Build in a fresh place so nothing from other projects interferes.
- **Build in order.** Each lesson adds one system and ends with a playtest. Don't skip the tests: horror bugs (a monster stuck on a wall) kill the fear instantly.

Every script below says where it lives in its first line, for example `-- ServerScriptService/MonsterAI`.

H.1 Atmosphere that scares

Players decide in the first 10 seconds whether your game is scary. That decision is made by **light and sound**, not by the monster.

Light: darkness with a shape

Pure black isn't scary, it's just frustrating. You want *readable* darkness: players can make out shapes, but not details.

1. Select **Lighting** and set **Technology** to **Future** in Properties (the only mode with realistic shadows from small lights).
2. Add this Script. It sets up the whole look in one place, so you can tweak numbers and press Play:

```

-- ServerScriptService/HorrorLighting
local Lighting = game:GetService("Lighting")

Lighting.ClockTime = 0
Lighting.Brightness = 0.4
Lighting.Ambient = Color3.fromRGB(8, 8, 12)
Lighting.OutdoorAmbient = Color3.fromRGB(12, 12, 18)
Lighting.EnvironmentDiffuseScale = 0.1
Lighting.EnvironmentSpecularScale = 0.3

-- Fog that eats the distance: players can't see what's at the end of a hallway
local atmosphere = Lighting:FindFirstChildOfClass("Atmosphere") or
Instance.new("Atmosphere")
atmosphere.Density = 0.45
atmosphere.Offset = 0
atmosphere.Color = Color3.fromRGB(40, 42, 55)
atmosphere.Decay = Color3.fromRGB(20, 20, 28)
atmosphere.Glare = 0
atmosphere.Haze = 2
atmosphere.Parent = Lighting

-- Cold, desaturated color grade
local grade = Lighting:FindFirstChild("HorrorGrade") or
Instance.new("ColorCorrectionEffect")
grade.Name = "HorrorGrade"
grade.Saturation = -0.35
grade.Contrast = 0.15
grade.TintColor = Color3.fromRGB(215, 225, 255)
grade.Parent = Lighting

```

3. Place a few **PointLights** in lamps with a warm color (255, 170, 110), **Range 12–16** and **Brightness 1–2**. Warm pools of light in a cold world are "safe zones". Players will run between them, and that's exactly the feeling you want.

Sound: half of all fear

1. In **SoundService**, set **AmbientReverb** to **StoneCorridor** (or **Hallway** for smaller buildings). Every footstep now echoes.
2. Add a **Sound** to **SoundService** named **Drone** : a low ambient loop, **Looped** on, **Volume 0.3**, **Playing** on. Search the Creator Store audio for "dark ambience" and use only audio uploaded by Roblox or by you.
3. For creepy one-off sounds (dripping, distant knocks), put a **Sound inside a Part**. Sounds inside parts are 3D: they get quieter with distance and come from a direction. Set **RollOffMaxDistance** to about

TIP Silence is a tool. A room with *no* ambient sound after a noisy hallway makes players nervous before anything happens.

Level design for fear

- **Limit sightlines.** Corners, doorways and T-junctions. Players should rarely see more than 30–40 studs ahead.
- **Build loops.** A chase is only fun if players can escape. Give every area a loop around a table, a shelf row or a block of rooms, so a skilled player can break line of sight.
- **Landmarks.** A red exit sign, a broken vending machine. Players need to remember where they are, or panic turns into confusion.
- **Contrast.** Alternate tight corridors with one bigger room. The change itself creates tension.

TRY IT

1. Build a small floor: 4–6 rooms, two corridors and one loop.
2. Add the lighting script, 5 warm lamps and the ambient drone.
3. Playtest alone with headphones. If it doesn't feel slightly uncomfortable yet, lower **Brightness** and add one more distant sound.

H.2 A flashlight with a battery

A flashlight turns darkness into gameplay: players *choose* where to look, and a draining battery adds pressure.

Build the tool:

1. In **StarterPack**, insert a **Tool** named **Flashlight**.
2. Inside it, add a Part named **Handle** (size **0.4, 0.4, 1.4**, dark metal).
3. Inside Handle, add a **SpotLight: Face Front, Angle 50, Range 40, Brightness 3, Shadows on, Enabled off**.
4. Optional: add a short click **Sound** to Handle named **Click**.

Now the logic. The server owns the battery, so it can't be hacked to infinite power:

```

-- StarterPack/Flashlight/FlashlightServer (Script)
local tool = script.Parent
local light = tool.Handle.SpotLight
local click = tool.Handle:FindFirstChild("Click")

local DRAIN_PER_SECOND = 1.5 -- battery % used per second while on
local RECHARGE_PER_SECOND = 0.5 -- slowly recovers while off
local BRIGHTNESS = 3

tool:SetAttribute("Battery", 100)

tool.Activated:Connect(function()
    if tool:GetAttribute("Battery") <= 0 then return end
    light.Enabled = not light.Enabled
    if click then click:Play() end
end)

tool.Unequipped:Connect(function()
    light.Enabled = false
end)

while true do
    local dt = task.wait(0.25)
    local battery = tool:GetAttribute("Battery")
    if light.Enabled then
        battery = math.max(0, battery - DRAIN_PER_SECOND * dt)
        if battery == 0 then light.Enabled = false end
        -- Flicker when the battery is low
        light.Brightness = if battery < 15 and math.random() < 0.25 then 0.3 else
BRIGHTNESS
    else
        battery = math.min(100, battery + RECHARGE_PER_SECOND * dt)
    end
    tool:SetAttribute("Battery", battery)
end

```

Show the battery. Create a ScreenGui named `HUD` in StarterGui with a TextLabel named `Battery` in a corner (**Visible** off). Then add a LocalScript inside the tool:

```
-- StarterPack/Flashlight/BatteryLabel (LocalScript)
local Players = game.GetService("Players")

local tool = script.Parent
local label = Players.LocalPlayer.PlayerGui.WaitForChild("HUD"):WaitForChild("Battery")

local function update()
    local battery = math.floor(tool:GetAttribute("Battery") or 0)
    label.Text = ("BATTERY %d%%"):format(battery)
    label.TextColor3 = if battery < 15 then Color3.fromRGB(255, 80, 80) else Color3.new(1,
1, 1)
end

tool:GetAttributeChangedSignal("Battery"):Connect(update)
tool.Equipped:Connect(function()
    label.Visible = true
    update()
end)
tool.Unequipped:Connect(function()
    label.Visible = false
end)
```

`tool.Activated` works on touch screens, mouse and gamepad, so mobile players can use the flashlight with no extra code.

WATCH OUT Keep **Shadows** on for the flashlight but off for most lamps. Many shadow-casting lights in one room cost a lot of performance on phones.

TRY IT

1. Playtest, equip the flashlight, click to toggle it.
2. Leave it on until the battery drops below 15% and watch it flicker.
3. Tune `DRAIN_PER_SECOND` so a full battery lasts about one minute of constant use.

H.3 A monster that hunts

The monster is the heart of the game. It needs three behaviors: **patrol** when it sees nobody, **chase** when it sees a player, and **give up** when it loses them. That last one matters: a monster that always knows where you are feels unfair, not scary.

Set it up:

1. **Avatar** → **Rig Builder**, make an R15 rig, name it **Monster** and give it a creepy look (tall, dark, thin). Make sure no part is **Anchored**.
2. Put it in **Workspace**. Add a looped breathing **Sound** and a **Sound** named **Scream** inside its **HumanoidRootPart**.
3. Create a Folder **PatrolPoints** in Workspace with 6–10 small parts spread over your map: **Anchored**, **CanCollide** off, **Transparency** 1.
4. In ReplicatedStorage, create a Folder **Remotes** with a **RemoteEvent** named **Jumpscare**. You'll use it in the next lesson.

```
-- ServerScriptService/MonsterAI
local Players = game:GetService("Players")
local PathfindingService = game:GetService("PathfindingService")
local ReplicatedStorage = game:GetService("ReplicatedStorage")

local monster = workspace:WaitForChild("Monster")
local humanoid = monster:WaitForChild("Humanoid")
local root = monster:WaitForChild("HumanoidRootPart")
local patrolPoints = workspace:WaitForChild("PatrolPoints"):GetChildren()
local jumpscare = ReplicatedStorage:WaitForChild("Remotes"):WaitForChild("Jumpscare")

local SIGHT_RANGE = 70
local CATCH_RANGE = 4
local PATROL_SPEED = 10
local CHASE_SPEED = 17 -- players walk at 16: they escape by using the map, not by
running
local GIVE_UP_AFTER = 4 -- seconds without seeing the target

root:SetNetworkOwner(nil) -- the server moves the monster, never a player's device

local path = PathfindingService:CreatePath({ AgentRadius = 2.5, AgentHeight = 6,
AgentCanJump = false })
local rayParams = RaycastParams.new()
rayParams.FilterType = Enum.RaycastFilterType.Exclude
rayParams.FilterDescendantsInstances = { monster }

local function canSee(character: Model): boolean
    local target = character:FindFirstChild("HumanoidRootPart")
    if not target then return false end
    local offset = target.Position - root.Position
    if offset.Magnitude > SIGHT_RANGE then return false end
    local hit = workspace:Raycast(root.Position, offset, rayParams)
    return hit ~= nil and hit.Instance:IsDescendantOf(character)
end
```

```

local function findTarget(): Model?
    local closest, closestDistance = nil, math.huge
    for _, player in Players:GetPlayers() do
        local character = player.Character
        local hum = character and character:FindFirstChildOfClass("Humanoid")
        if hum and hum.Health > 0 and canSee(character) then
            local distance = (character.HumanoidRootPart.Position -
root.Position).Magnitude
            if distance < closestDistance then
                closest, closestDistance = character, distance
            end
        end
    end
    return closest
end

local function catch(character: Model)
    local player = Players:GetPlayerFromCharacter(character)
    local hum = character:FindFirstChildOfClass("Humanoid")
    if not player or not hum then return end
    hum.WalkSpeed = 0 -- freeze the victim
    jumpscare:FireClient(player) -- the scare plays on their screen only
    task.wait(1.2) -- let it land before the reset
    hum.Health = 0
end

local function walkTo(position: Vector3, stopIf: () -> boolean)
    local ok = pcall(path.ComputeAsync, path, root.Position, position)
    if not ok or path.Status ~= Enum.PathStatus.Success then return end
    for _, waypoint in path:GetWaypoints() do
        if stopIf() then return end
        humanoid:MoveTo(waypoint.Position)
        humanoid.MoveToFinished:Wait()
    end
end

local function chase(character: Model)
    humanoid.WalkSpeed = CHASE_SPEED
    local scream = root:FindFirstChild("Scream")
    if scream then scream:Play() end
    local lostAt = nil

    while character.Parent do

```

```

    local targetRoot = character:FindFirstChild("HumanoidRootPart")
    local targetHum = character:FindFirstChildOfClass("Humanoid")
    if not targetRoot or not targetHum or targetHum.Health <= 0 then return end

    if (targetRoot.Position - root.Position).Magnitude <= CATCH_RANGE then
        catch(character)
        return
    end

    if canSee(character) then
        lostAt = nil
        humanoid:MoveTo(targetRoot.Position) -- in sight: run straight at them
    else
        lostAt = lostAt or os.clock()
        if os.clock() - lostAt > GIVE_UP_AFTER then return end
        -- out of sight: follow a path to where they are now
        local ok = pcall(path.ComputeAsync, path, root.Position, targetRoot.Position)
        local waypoints = ok and path.Status == Enum.PathStatus.Success and
path:GetWaypoints()
        if waypoints and waypoints[2] then
            humanoid:MoveTo(waypoints[2].Position)
        end
    end
    task.wait(0.15)
end
end

while true do
    local target = findTarget()
    if target then
        chase(target)
    else
        humanoid.WalkSpeed = PATROL_SPEED
        local point = patrolPoints[math.random(#patrolPoints)]
        walkTo(point.Position, function()
            return findTarget() ~= nil
        end)
    end
    task.wait(0.1)
end
end

```

How it thinks:

- `canSee` fires a ray from the monster to the player. If the first thing it hits is the player, there's a clear line of sight. Walls, doors and shelves block it.
- While chasing, the monster runs straight at a visible player, and follows a computed path around corners when it can't see them.
- After 4 seconds without sight it gives up and goes back to patrolling. That's the moment players *earn* by hiding well.

TIP `CHASE_SPEED` 17 vs. player speed 16 is deliberate. On a straight line the monster slowly wins, so players must use loops and doors. Test different values, but keep the monster only slightly faster.

WATCH OUT If the monster gets stuck on corners, increase `AgentRadius` and make doorways at least 6 studs wide. Pathfinding also ignores parts with `CanCollide` off, so the monster won't walk around decorations you can walk through.

TRY IT

1. Playtest with **Test → Clients and Servers** (2 players).
2. Walk into the monster's view: it should scream and chase.
3. Break line of sight around a corner and hide. After about 4 seconds it should give up.
4. Get caught once: you'll freeze and reset (the jumpscare itself comes next).



Jumpscares that don't lag

A jumpscare fails if the image appears half a second late or the sound stutters. The fix is simple: **preload** the image and sound on the client when the player joins, and play the scare locally.

Build the UI:

1. In StarterGui, create a ScreenGui `JumpscareGui`: `IgnoreGuiInset` on, `ResetOnSpawn` off, `DisplayOrder` 100.
2. Inside, add an ImageLabel `Face`: `AnchorPoint` `0.5, 0.5`, `Position` `{0.5,0},{0.5,0}`, `Size` `{1,0},{1,0}`, `BackgroundTransparency` 1, `Visible` off. Set `Image` to your monster close-up (upload it via the Asset Manager).
3. Inside the ScreenGui, add a Sound `Scream` (a short, loud sting, Volume 1).

```
-- StarterPlayerScripts/Jumpscare (LocalScript)
local ContentProvider = game:GetService("ContentProvider")
local Players = game:GetService("Players")
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local RunService = game:GetService("RunService")
```

```

local TweenService = game:GetService("TweenService")

local gui = Players.LocalPlayer:WaitForChild("PlayerGui"):WaitForChild("JumpscareGui")
local face = gui:WaitForChild("Face")
local scream = gui:WaitForChild("Scream")
local jumpscare = ReplicatedStorage:WaitForChild("Remotes"):WaitForChild("Jumpscare")
local camera = workspace.CurrentCamera

-- Download the image and the sound now, so the scare is instant later.
task.spawn(function()
    ContentProvider:PreloadAsync({ face, scream })
end)

local function shake(duration: number, strength: number)
    local start = os.clock()
    RunService:BindToRenderStep("JumpscareShake", Enum.RenderPriority.Camera.Value + 1,
function()
    local t = os.clock() - start
    if t >= duration then
        RunService:UnbindFromRenderStep("JumpscareShake")
        return
    end
    local power = strength * (1 - t / duration)
    camera.CFrame *= CFrame.Angles(
        math.rad((math.random() - 0.5) * power),
        math.rad((math.random() - 0.5) * power),
        0
    )
end)
end

jumpscare.OnClientEvent:Connect(function()
    face.ImageTransparency = 0
    face.Size = UDim2.fromScale(1.4, 1.4)
    face.Visible = true
    scream:Play()
    TweenService:Create(face, TweenInfo.new(0.15, Enum.EasingStyle.Back), { Size =
UDim2.fromScale(1, 1) }):Play()
    shake(0.7, 8)
    task.wait(0.8)
    TweenService:Create(face, TweenInfo.new(0.4), { ImageTransparency = 1 }):Play()
    task.wait(0.4)
end)

```

```
face.Visible = false
end)
```

`ResetOnSpawn` off is important: the player dies right after the scare, and a resetting GUI would cut it off.

Scripted scares: the ones between chases

Big jumpscares should be rare. Most fear comes from small events: a door slams behind you, the lights flicker, footsteps above. These are cosmetic, so they can run entirely on the client.

1. Create a Folder `ScareZones` in Workspace.
2. Add trigger parts: **Anchored**, **CanCollide** off, **Transparency** 1. Put a **Sound** inside each (slam, whisper, footsteps).
3. Optional: add an **ObjectValue** named `Lights` in a zone, pointing at a Model or Folder with lamps to flicker.

```

-- StarterPlayerScripts/ScareZones (LocalScript)
local Players = game:GetService("Players")

local player = Players.LocalPlayer
local triggered = {}

local function flicker(target: Instance)
    for _ = 1, 6 do -- an even number, so the lights end as they started
        for _, light in target:GetDescendants() do
            if light:IsA("Light") then
                light.Enabled = not light.Enabled
            end
        end
        task.wait(0.08 + math.random() * 0.12)
    end
end

for _, zone in workspace:WaitForChild("ScareZones"):GetChildren() do
    zone.Touched:Connect(function(hit)
        if triggered[zone] then return end
        local character = player.Character
        if not character or not hit:IsDescendantOf(character) then return end
        triggered[zone] = true

        local sound = zone:FindFirstChildOfClass("Sound")
        if sound then sound:Play() end
        local lights = zone:FindFirstChild("Lights")
        if lights and lights.Value then
            flicker(lights.Value)
        end
    end)
end
end

```

Each zone fires once per player per server, so the scare never gets stale.

Rules for good scares:

- **Anticipation beats the scare.** Flicker the lights, *then* make players wait 3 seconds before anything happens.
- **Space them out.** One small scare every 60–90 seconds, one big one per round at most.
- **Never loop the same scare.** Once a player has seen it, it's furniture.

TRY IT

1. Add 3 scare zones along the main route: a door slam, a flicker, distant footsteps.
2. Playtest and get caught: the face should appear instantly with the sound and shake.
3. Test on a phone (or the device emulator) and check the face covers the screen.

H.5 Objective, escape and rounds

Fear needs a goal. Without one, players hide in a corner forever. In *Night Shift*, players collect **5 fuses**, power the **generator** and escape through the **exit door** while the monster hunts them.

Build it:

1. In ServerStorage, create a Part named **Fuse** (small, yellow, Neon, **Anchored**) with a **ProximityPrompt** inside (ActionText **Pick up**, HoldDuration **0.5**).
2. In Workspace, add a Folder **FuseSpots** with 10+ invisible anchored parts where fuses may appear, and an empty Folder **Fuses**.
3. Add a Part **Generator** with a ProximityPrompt (ActionText **Insert fuses**, HoldDuration **1**).
4. Add a Part **ExitDoor** (the door) and an invisible Part **ExitZone** behind it (**CanCollide** off).

```
-- ServerScriptService/Objectives
local Players = game:GetService("Players")
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local ServerStorage = game:GetService("ServerStorage")
local TweenService = game:GetService("TweenService")

local FUSES_NEEDED = 5
local EXTRA_FUSES = 2 -- a little slack so the round is never unwinnable

local fuseTemplate = ServerStorage:WaitForChild("Fuse")
local spots = workspace:WaitForChild("FuseSpots"):GetChildren()
local fuseFolder = workspace:WaitForChild("Fuses")
local generator = workspace:WaitForChild("Generator")
local exitDoor = workspace:WaitForChild("ExitDoor")
local exitZone = workspace:WaitForChild("ExitZone")

local placed = 0

local function spawnFuse(cframe: CFrame)
    local fuse = fuseTemplate:Clone()
    fuse.CFrame = cframe
    fuse.Parent = fuseFolder
    fuse.ProximityPrompt.Triggered:Connect(function(player)
```

```

        if not fuse.Parent then return end -- someone was faster
        fuse:Destroy()
        player:SetAttribute("Fuses", (player:GetAttribute("Fuses") or 0) + 1)
    end)
end

local function randomSpot(): CFrame
    return spots[math.random(#spots)].CFrame
end

local function openExit()
    ReplicatedStorage:SetAttribute("ExitOpen", true)
    exitDoor.CanCollide = false
    TweenService:Create(exitDoor, TweenInfo.new(1.5), { Transparency = 1 }):Play()
end

-- Call this at the start of every round
local function resetRound()
    fuseFolder:ClearAllChildren()
    placed = 0
    ReplicatedStorage:SetAttribute("FusesPlaced", 0)
    ReplicatedStorage:SetAttribute("ExitOpen", false)
    exitDoor.CanCollide = true
    exitDoor.Transparency = 0
    for _, player in Players:GetPlayers() do
        player:SetAttribute("Fuses", 0)
        player:SetAttribute("Escaped", false)
    end
    -- shuffle the spots, then use the first few
    local shuffled = table.clone(spots)
    for i = #shuffled, 2, -1 do
        local j = math.random(i)
        shuffled[i], shuffled[j] = shuffled[j], shuffled[i]
    end
    for i = 1, math.min(FUSES_NEEDED + EXTRA_FUSES, #shuffled) do
        spawnFuse(shuffled[i].CFrame)
    end
end

generator.ProximityPrompt.Triggered:Connect(function(player)
    local held = player:GetAttribute("Fuses") or 0
    if held == 0 then return end
    player:SetAttribute("Fuses", 0)
    placed += held
end)

```

```

    ReplicatedStorage:SetAttribute("FusesPlaced", math.min(placed, FUSES_NEEDED))
    if placed >= FUSES_NEEDED and not ReplicatedStorage:GetAttribute("ExitOpen") then
        openExit()
    end
end)

exitZone.Touched:Connect(function(hit)
    if not ReplicatedStorage:GetAttribute("ExitOpen") then return end
    local player = Players:GetPlayerFromCharacter(hit.Parent)
    if not player or player:GetAttribute("Escaped") then return end
    player:SetAttribute("Escaped", true)
    -- Reward here, e.g. with PlayerDataService from the Core Course
end)

-- A caught player drops their fuses somewhere on the map, so nothing is lost for the
team
Players.PlayerAdded:Connect(function(player)
    player.CharacterAdded:Connect(function(character)
        character:WaitForChild("Humanoid").Died:Connect(function()
            local held = player:GetAttribute("Fuses") or 0
            player:SetAttribute("Fuses", 0)
            for _ = 1, held do
                spawnFuse(randomSpot())
            end
        end)
    end)
end)
end)

resetRound()

```

The HUD. Show the team's progress with a TextLabel that reads the attributes:

```

-- StarterGui/HUD/Objective (LocalScript, next to a TextLabel named ObjectiveLabel)
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local label = script.Parent.WaitForChild("ObjectiveLabel")

local function update()
    if ReplicatedStorage:GetAttribute("ExitOpen") then
        label.Text = "THE EXIT IS OPEN. RUN."
    else
        label.Text = ("Fuses in generator: %d / 5"):format(ReplicatedStorage:GetAttribute("FusesPlaced") or 0)
    end
end

ReplicatedStorage:GetAttributeChangedSignal("FusesPlaced"):Connect(update)
ReplicatedStorage:GetAttributeChangedSignal("ExitOpen"):Connect(update)
update()

```

Rounds

Use the round loop from Core Course Module 7: an intermission in a safe lobby, then a round of 6–8 minutes. Call `resetRound()` at the start of each round (move it into a ModuleScript if the round manager lives in another script), teleport everyone in, and end the round when all living players have escaped or time runs out. Also teleport the monster back to a patrol point at the start of each round.

Fair monetization for horror

Horror players hate anything that breaks the fear. Sell things that *add* to it:

Idea	Type	Why it's fair
Flashlight skins (colored beams, lantern)	Game Pass	Pure cosmetics
Revive (continue as a ghost that can open doors)	Developer Product	Keeps the player in the round with friends
Private servers	Built-in	Friends want to play together
Extra battery	Avoid	Directly weakens the core tension

Grant every developer product in `ProcessReceipt` only, exactly as in Core Course Module 8.

Launch tip

Horror spreads through **reaction clips**. Record your friends' first playthrough (with permission) and post the best 15-second scream as a TikTok, Reel and Short. Put the game's name on screen in the first second.

TRY IT

1. Play a full round with 2 players: collect fuses, get one player caught, check that their fuses reappear.
2. Insert 5 fuses: the exit should open and the HUD should change.
3. Escape through the door.

Your Night Shift checklist

- ☐ Future lighting, fog, color grade and warm "safe" lamps
- ☐ Ambient drone, reverb and 3D spot sounds
- ☐ Flashlight with a server-side battery and HUD
- ☐ Monster that patrols, chases on sight and gives up
- ☐ Preloaded jumpscare with sound and camera shake
- ☐ 3+ one-time scare zones along the route
- ☐ Fuses, generator, exit and a working HUD
- ☐ Tested with 2 players and on a phone